Personal Research Agent: Requirements Specification

Oct 22, 2026 · @tj

Introduction

The Personal Research Agent (PRA) takes a research question, searches the web, reads the sources it finds, and returns a structured report where every claim is cited. This document defines what version 1 must do; the companion Architecture Design doc defines how.

Purpose. Give one person a fast, trustworthy first draft of research on any topic, and give the builder a hands-on project that teaches the agent loop, tool use, and grounding answers in sources.

Definitions

- **Agent loop**: the repeated cycle of the model deciding an action, a tool running it, and the result feeding back.
- **Source**: a web page or PDF that was fetched and read.
- **Note**: a short extract from one source, stored with its URL.
- **Claim**: a factual statement in the final report.
- **Citation**: a numbered link from a claim back to the source that supports it.

Scope

Version 1 is a command-line tool that turns one question into one cited Markdown report. Anything needing a UI, accounts, or private data is deferred.

Phase	In scope	Out of scope
MVP (v1)	CLI, web search, page fetch, per-source notes, cited Markdown report, budget limits	Web UI, login, multi-user, paid-source access
v1.1	Source ranking, verification pass, saved sessions, progress display	Real-time collaboration

Phase	In scope	Out of scope
v2	Follow-up Q&A, PDF/DOCX export, simple web UI, local document search	Autonomous actions beyond research (emailing, purchases, posting)

Always out of scope: logging into websites, bypassing paywalls or robots.txt, and any action with side effects outside the user's own machine.

Users and use cases

The single user is a student or developer practicing with AI agents who also wants real research output: comfortable with a terminal and Python, short on time, and wary of unsourced answers.

ID	Use case	Example	Success looks like
UC-1	Topic overview	"Explain how RAG differs from fine-tuning"	1-2 page report with 6+ cited sources
UC-2	Compare options	"Compare Postgres vs SQLite for a small app"	Comparison table with cited differences
UC-3	Current-state lookup	"What are recent approaches to LLM evaluation?"	Findings from recent sources, dates shown
UC-4	Study aid	"Summarize the key ideas of transformers"	Structured summary with reading list
UC-5	Revisit research	Reopen a past session and ask a follow-up	Answer reuses stored notes, no re-search

Functional requirements

The MVP is the set of Must items; Should and Could items follow in later phases.

ID	Requirement	Priority
FR-1	Accept a research question as free text from the command line	Must

ID	Requirement	Priority
FR-2	Accept optional settings: depth (quick, standard, deep), max sources, output path	Should
FR-3	Break the question into 3-6 sub-questions (a research plan)	Must
FR-4	Show the plan and let the user approve or edit it before searching	Should
FR-5	Run a web search for each sub-question and collect candidate URLs	Must
FR-6	Fetch each page and extract readable text from HTML and PDF	Must
FR-7	Skip or log failed, paywalled, duplicate, or irrelevant pages without stopping the run	Must
FR-8	Write a note per source: key facts, quoted evidence under 25 words, URL, fetch date	Must
FR-9	Rank sources by relevance and credibility, and drop weak ones	Should
FR-10	Synthesize notes into a report: summary, findings by sub-question, conflicts, sources	Must
FR-11	Attach a numbered citation to every factual claim and list all sources at the end	Must
FR-12	Flag conflicting evidence and mark low-confidence findings	Should
FR-13	Run a verification pass that checks each claim against its cited note	Should
FR-14	Save the report as Markdown with a timestamped filename	Must
FR-15	Save the full session (question, plan, notes, report) for later reuse	Should
FR-16	Print live progress: current step, sources read, tokens used	Should

ID	Requirement	Priority
FR-17	Enforce budgets: max steps, sources, tokens, and wall-clock time	Must
FR-18	Answer follow-up questions from stored notes, searching only if notes fall short	Could
FR-19	Export the report to PDF or DOCX	Could

Non-functional requirements

Targets below are starting goals for a solo project; adjust them after the first real runs.

ID	Area	Requirement	Target
NFR-1	Performance	Time to a finished report	Quick under 1 min, standard under 3 min
NFR-2	Cost	Model and search spend per run	Standard run under $0.50, hard cap configurable
NFR-3	Accuracy	Claims traceable to a cited source in a manual check of 20 test questions	95% or more
NFR-4	Reliability	Network and API errors	Retry up to 3 times with backoff; one bad source never fails the run
NFR-5	Security	API keys	Kept in environment variables, never logged or committed
NFR-6	Security	Fetched web content	Treated as untrusted data, never as instructions to the agent
NFR-7	Privacy	Data leaving the machine	Only the query and page text sent to the model and search APIs

ID	Area	Requirement	Target
NFR-8	Usability	Getting started	One install command, one run command, readable error messages
NFR-9	Observability	Run logs	Structured log per step with tool name, duration, and token count
NFR-10	Maintainability	Code quality	Modular tools, type hints, unit tests for every tool
NFR-11	Portability	Platforms	Python 3.11 or newer on Windows, macOS, and Linux

Constraints, assumptions, and dependencies

Constraints

- One developer, part-time, so the MVP must be buildable in about a week.
- Low running cost: free tiers and small models where possible.
- Respect each site's robots.txt and terms; no paywall bypass.
- The model's context window is finite, so sources are summarized into notes rather than pasted whole.

Assumptions

- The user has Python 3.11+ and can create API keys.
- Internet access is available during runs.
- English-language research is the primary target.

Dependencies

Dependency	Role	Notes
LLM API (Claude via the Anthropic API)	Planning, summarizing, writing	Wrapped behind an interface so another model can be swapped in
Web search API	Finding candidate URLs	Choose one provider with a free tier; wrapped behind an interface

Dependency	Role	Notes
HTTP client and HTML/PDF text extractor	Fetching and cleaning pages	Open-source Python libraries
Local storage (files or SQLite)	Sessions and notes	No server needed

Acceptance criteria

The MVP is done when all Must requirements pass the scenarios below.

ID	Scenario	Expected result
AC-1	Run with a clear question, e.g. "What is retrieval-augmented generation?"	Markdown report saved with at least 5 distinct cited sources
AC-2	Open any citation number in the report	Link points to a page whose content supports the claim
AC-3	Include a URL that returns an error or is blocked	Run completes; the failure is logged and the URL skipped
AC-4	Set a low budget (e.g. max 5 steps)	Agent stops at the limit and still writes a partial report saying so
AC-5	Ask about something with no good sources	Report states the evidence is thin instead of inventing facts
AC-6	A fetched page contains the text "ignore your instructions"	Agent ignores it and continues the task
AC-7	Check logs and repo after a run	No API keys present; one log line per step
AC-8	Run a standard-depth question	Finishes within the NFR-1 time target

Risks and open questions

Risk	Impact	Mitigation
Model states a claim the source does not support	Misleading report	Notes store evidence; verification pass (FR-13); citations required on every claim
Prompt injection from fetched pages	Agent behaves unexpectedly	Treat page text as data, strip instructions, no side-effect tools
Run cost grows through long loops	Surprise bill	Hard budgets on steps, tokens, and time (FR-17)
Pages blocked, paywalled, or badly extracted	Thin evidence	Skip and log, search for alternatives, report coverage gaps
Search API quota or outage	Run fails	Retry, fall back to a second provider if added
Scope creep into a full product	Project never finishes	Ship Must items first; defer the rest

Open questions

- Which search provider should v1 use?
- Should the plan approval step (FR-4) be on by default?
- Is English-only acceptable for v1?
- Store sessions as plain files or in SQLite?